

CW1 Group Report | 3D Interactivity Media Product

Planning for CW2: 3DUI Interactivity Report

Item	Details
Module	5MMCS007W – 3D Interactive Media Product
Group Number	Group No.05
Project title	Lost Island | Island Treasure Hunter
Group Members	Sanuka Udan - 20240970 | w2181823 Savindi Nawarthana – 20241374 | w2181807 Hasiru Yasod- 20240969 | w2181832 Rashen Nimsara - 20240724 | w2181815
GitHub repo	https://github.com/rashennimsara/TreasureIsland-GroupProject
Course Work Submission Date	Thursday 12th November of 2026 at 18:30:00

1.Overview

1.1 Concept and Design

1.1.1 Research and Reference

Before developing Island Treasure Hunter, our group explored different LEGO-themed games, tropical island environments and Unity tutorials to understand suitable design and interaction techniques. The research focused on creating an engaging 3D environment with interactive objects, character movement and gameplay challenges. Visual references helped us plan the island's layout, including the beach, forest, mountain and waterfall, while Unity tutorials provided guidance on mouse interaction, UI development, animations and collision-based triggers. These references supported our initial design decisions and helped us identify features to develop during CW1 and improve further in CW2.

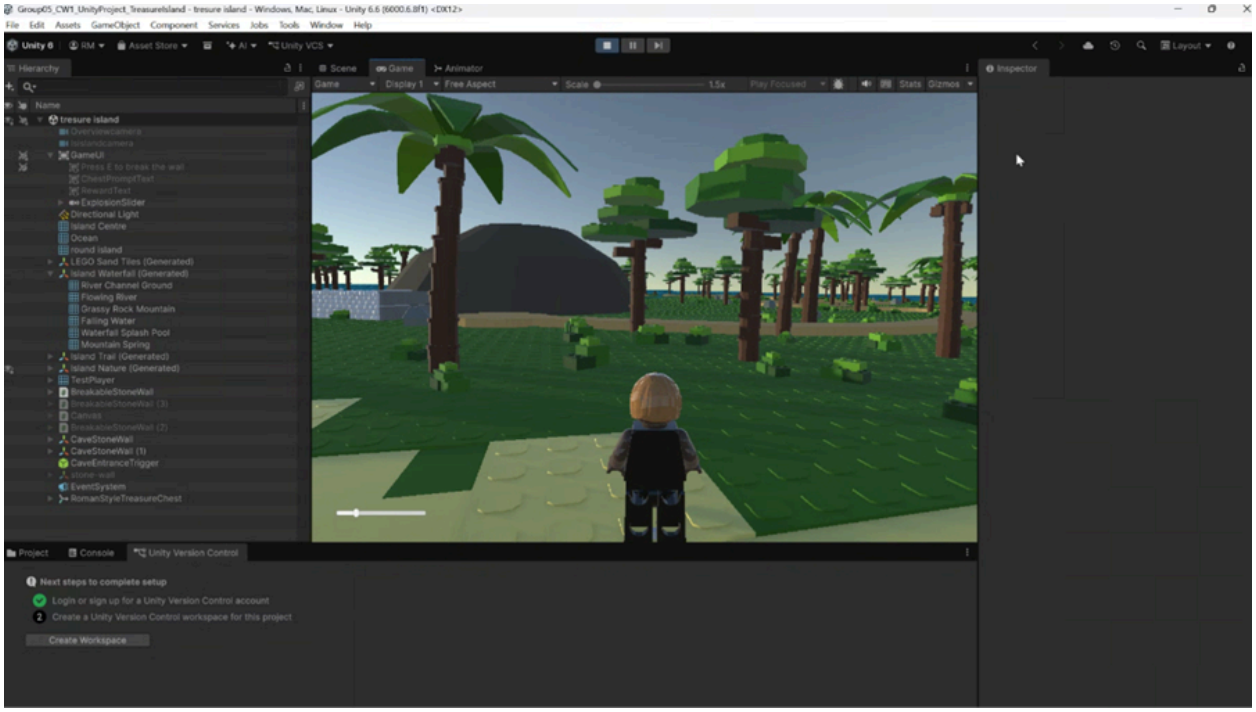
Reference	Possible design relevance	Source / URL
LEGO pirate / island models	Brick-built landscape and vegetation	https://kenney.nl/assets/brick-kit?utm
Treasure exploration game or film	Island journey and discovery loop	https://assetstore.unity.com/packages/3d/props/interior/animated-pbr-chest-demo-194755
Unity Tutorials 02–05	Input, colliders, triggers and UI	https://learning.westminster.ac.uk/ultra/courses/_106775_1/document/_6256529_1?view=content&state=view
Group concept images	Forest route, river crossing and terrain	https://danielilett.com/2020-04-05-tut5-3-urp-stylised-water/

1.1.2 Problem and Project Concept

The main challenge addressed by Island Treasure Hunter is designing an engaging three-dimensional adventure that combines exploration, object manipulation and interactive gameplay. The project aims to demonstrate how different interaction techniques can work together in a LEGO-style virtual environment. The group selected a tropical island setting because it provides opportunities for environmental exploration and obstacle-based challenges. The island includes a sandy beach, forest, mountain, river and waterfall. The intended player character is a LEGO minifigure who explores the environment, interacts with objects and progresses towards hidden treasure. The gameplay design combines mouse-based object manipulation, keyboard-controlled movement, menu interactions and collision-based triggers. During CW1, the group developed the island environment and tested several components separately, including draggable LEGO objects and a cave entrance trigger. The main objective for CW2 is to integrate these systems into a playable adventure, introduce additional challenges and provide clear feedback as players progress towards the treasure.

1.1.3 Basic Design

The basic design of Island Treasure Hunter follows a 3D diorama concept, presenting a colorful LEGO-style tropical island where players can explore and interact with different environmental objects. The island was created in Unity using a flattened cylinder to form a circular landscape, with sand-colored tiles along the coastline and green LEGO-style tiles covering the interior. To create a realistic tropical atmosphere while maintaining the LEGO theme, the environment includes palm trees, bushes, rocks, a mountain, an animated waterfall, a river and surrounding ocean water. The gameplay layout is designed to guide the player from the beach through the forest towards different obstacles and treasure-hunting challenges. The interaction design combines mouse-controlled object manipulation, keyboard-based character movement, UI menus and collision triggers. During CW1, the group developed the environment and tested individual interaction components, while further integration, animations and gameplay improvements are planned for CW2.

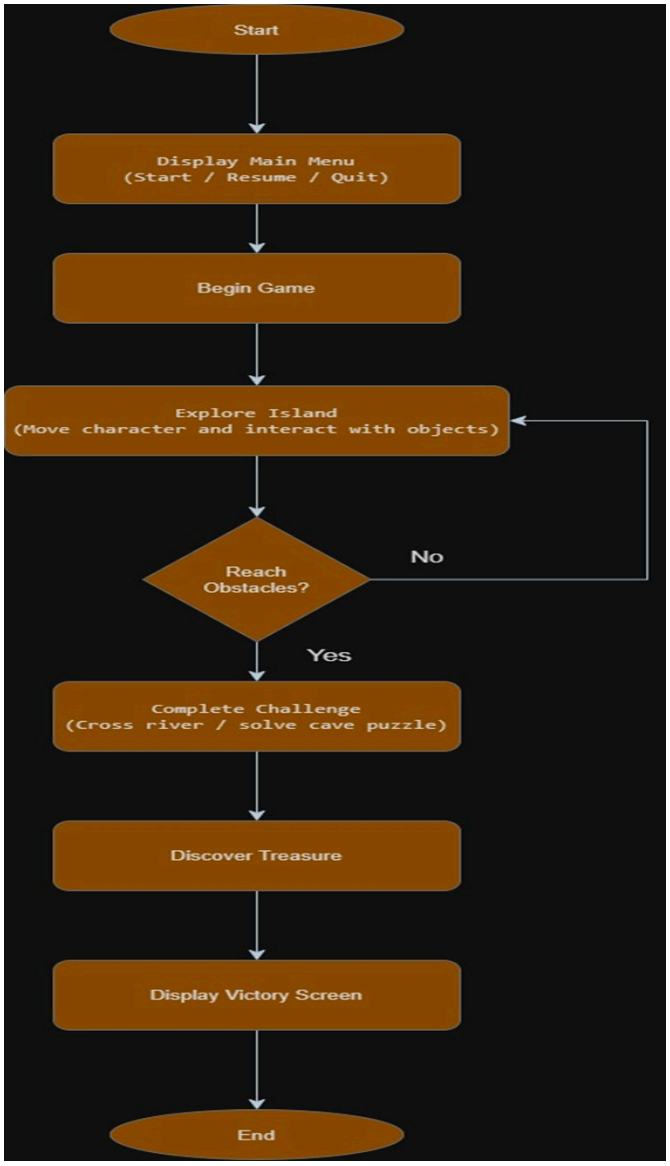


1.1.4 3DUI Theory and Concepts Used

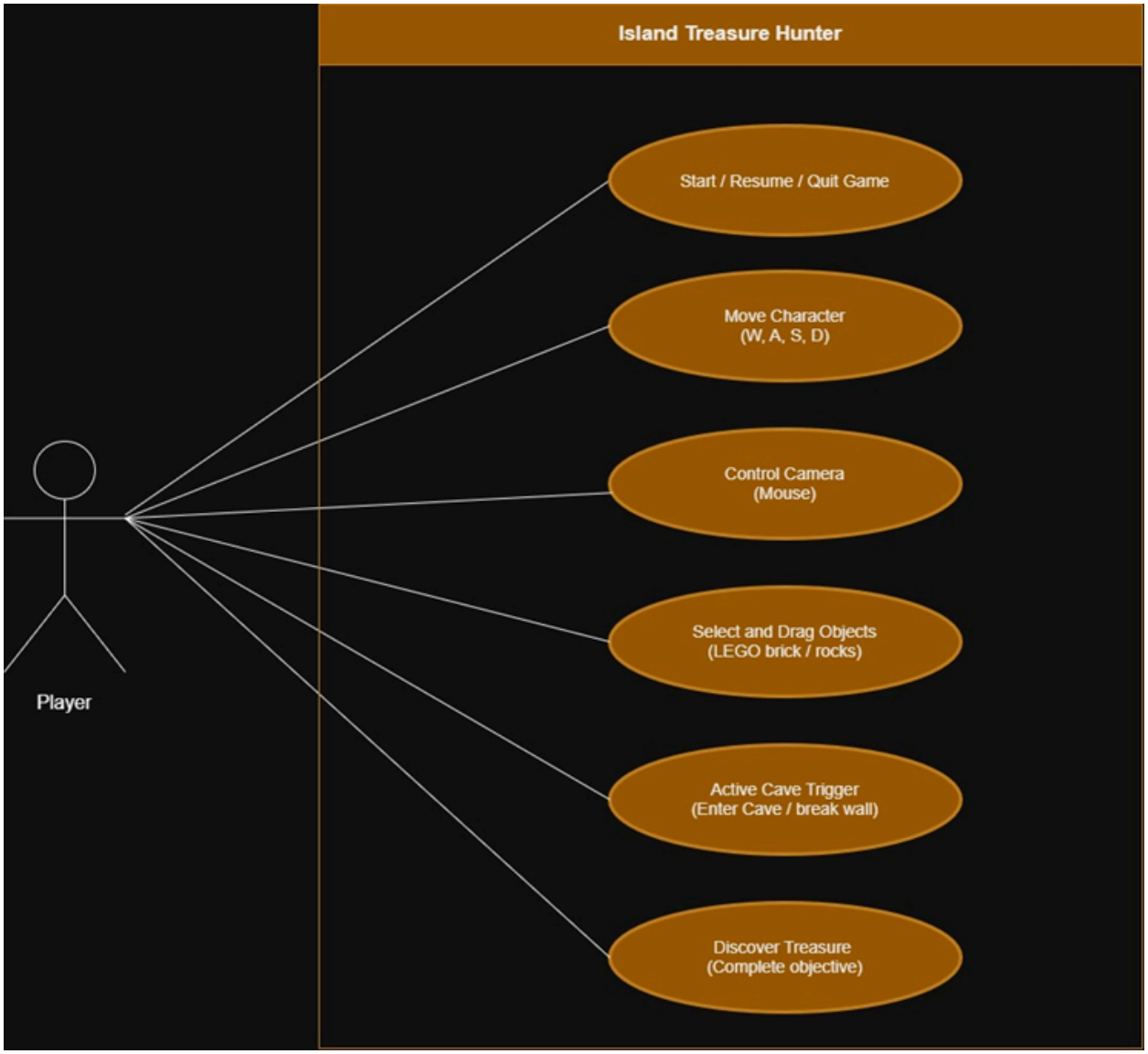
3DUI concept	How it is applied in our project
Selection and manipulation	OnMouseDrag LEGO brick and rock — tested in Student 2 test scene
Navigation	Third-person player and camera scripts prepared; final-character integration unverified
Wayfinding	Coastline, river, forest route and mountain as visual landmarks
System control	Canvas Start/Resume/Quit menu; functionality requires full button testing
Feedback	Visible object movement; cave-trigger Console detection reported by Member 4
Triggers / Physics	Member 4 cave trigger and breakable wall; wall explosion result unconfirmed

1.2 Design Diagrams

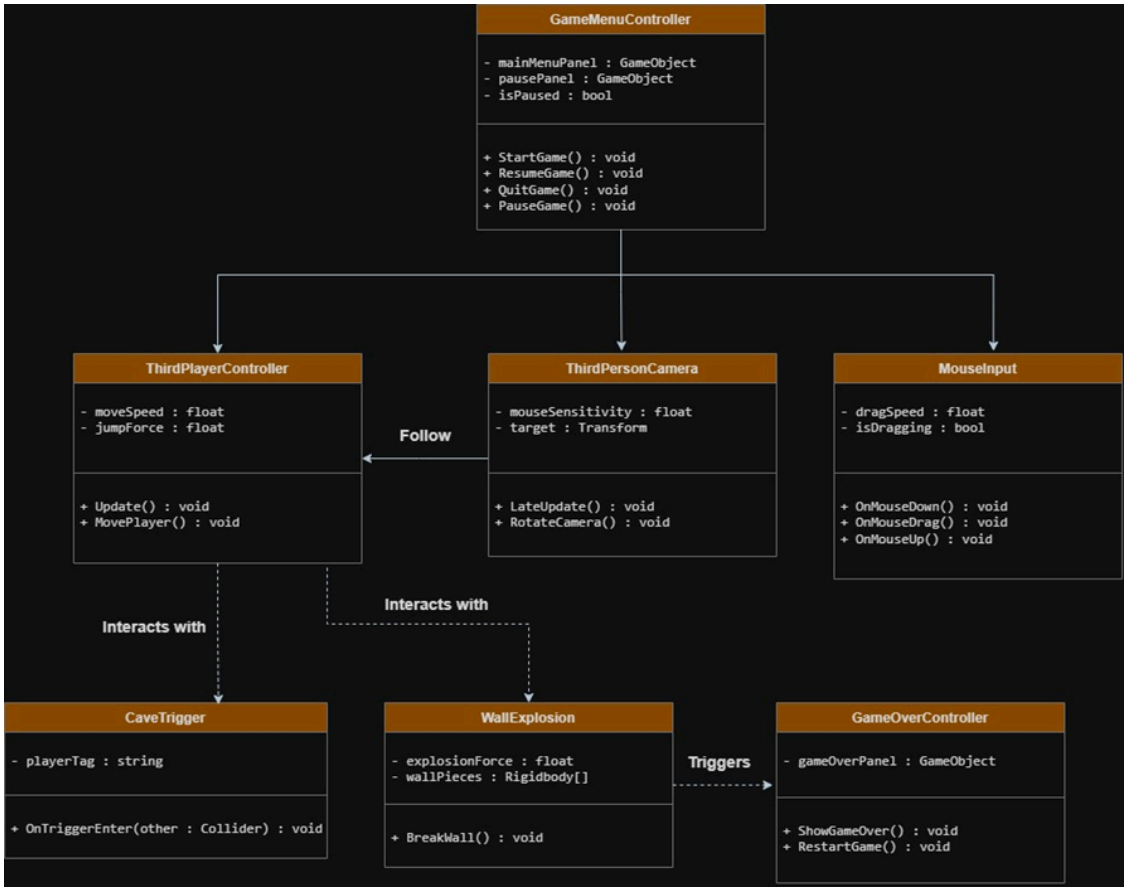
1.2.1 Storyboard / Flowchart



1.2.2 Basic Use Case Diagram



1.2.3 Basic Domain / Class Diagram



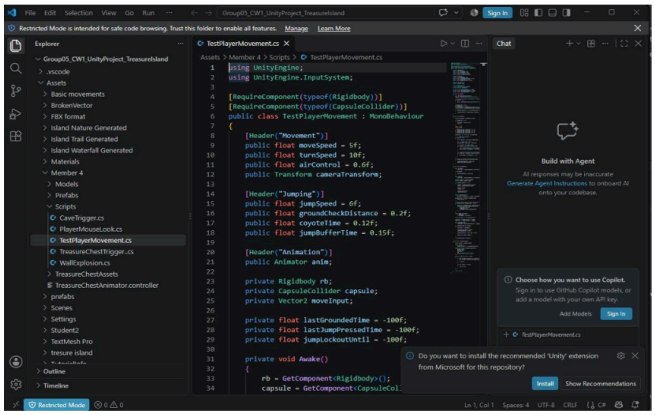
Design Area	Classes / scripts	Responsibility
Scene / Design	Component / script	Group environment assets
View / System control	Island Scene / Ground / River	Menu and game-over panels in Student 2 test scene
Interaction	GameMenuController; GameOverController	Mouse drag applied to brick and rock; tested
Navigation	MouseInput	Prepared; final character/camera test pending
Triggers / Animation	ThirdPersonPlayerController; ThirdPersonMouseCamera	Member 4 work; verify full physics response

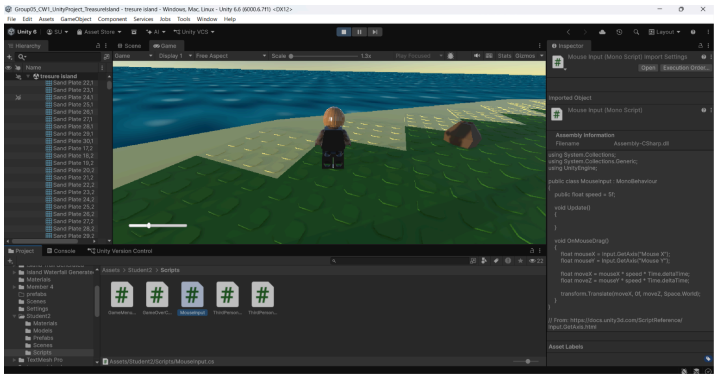
2. Navigation Framework

Control or View	What it does in Island Treasure Hunter
WASD	Moves the LEGO minifigure along the road and around obstacles.
Space	Makes the minifigure jump over obstacles.
Mouse movement	Steers the minifigure in the direction the mouse moves.
Third person camera	Shows the minifigure and the surrounding route.
E	Opens the treasure chest or the wall.

Route and mission failure

- **Route:** The player starts on the beach and follows the road across the river. The road then continues behind the rock.
- **Obstacles and failure:** The player steers around obstacles or jumps over them. The mission fails if the minifigure falls into the sea or river, or fails to get past an obstacle.





3. Interactivity | Selection, Manipulation and Triggers

The interactivity of Island Treasure Hunter focuses on allowing players to explore the 3D environment and interact with different objects using mouse and keyboard controls. The project combines object selection, direct manipulation, UI buttons and collision-based triggers to create an engaging gameplay experience. These interaction techniques are designed to help players overcome obstacles, explore the island and progress towards discovering hidden treasure

3.1 World-Space UI

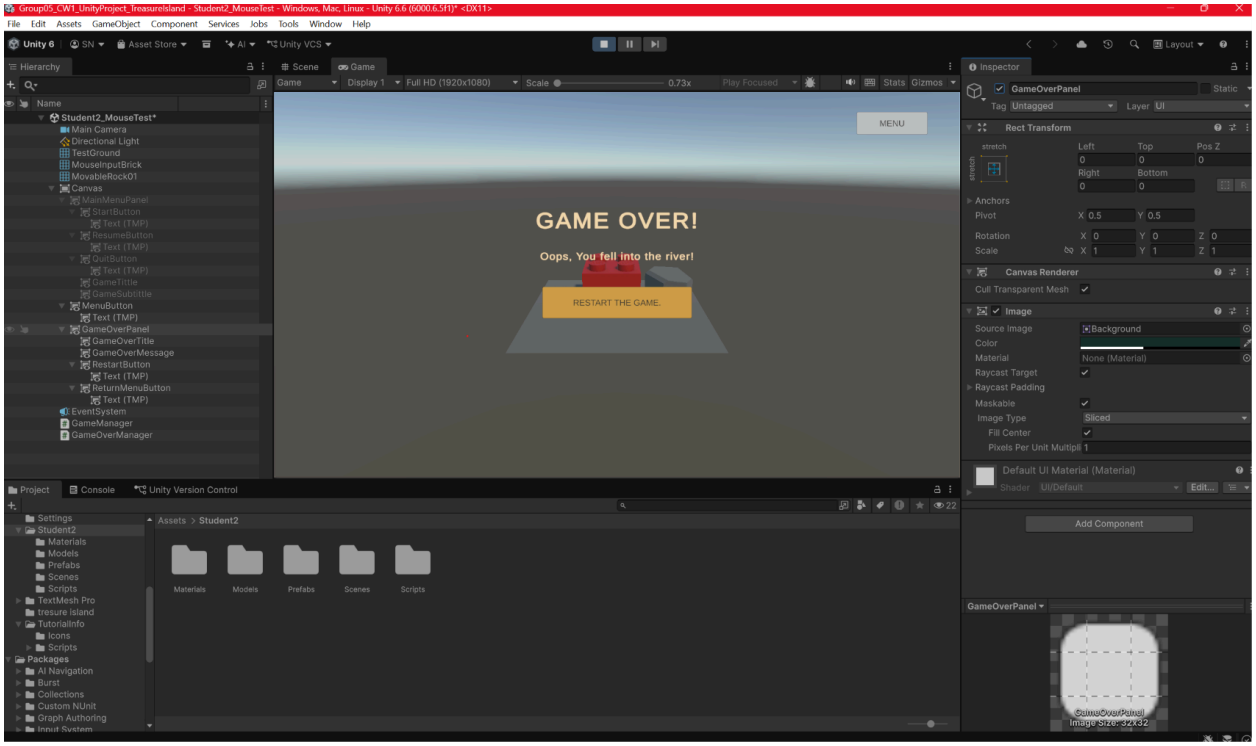
The project uses Unity's Canvas system to create interactive buttons and menus that allow players to control different gameplay functions. During CW1, a screen-space Canvas was developed with Start, Resume and Quit buttons, managed through the GameMenuController.cs script. For CW2, the group plans to introduce world-space UI elements, such as an interactive button near a treasure chest or cave entrance. By setting the Canvas Render Mode to World Space, these buttons can appear directly within the 3D environment rather than remaining fixed on the screen. This approach would improve immersion and allow players to interact with objects more naturally.

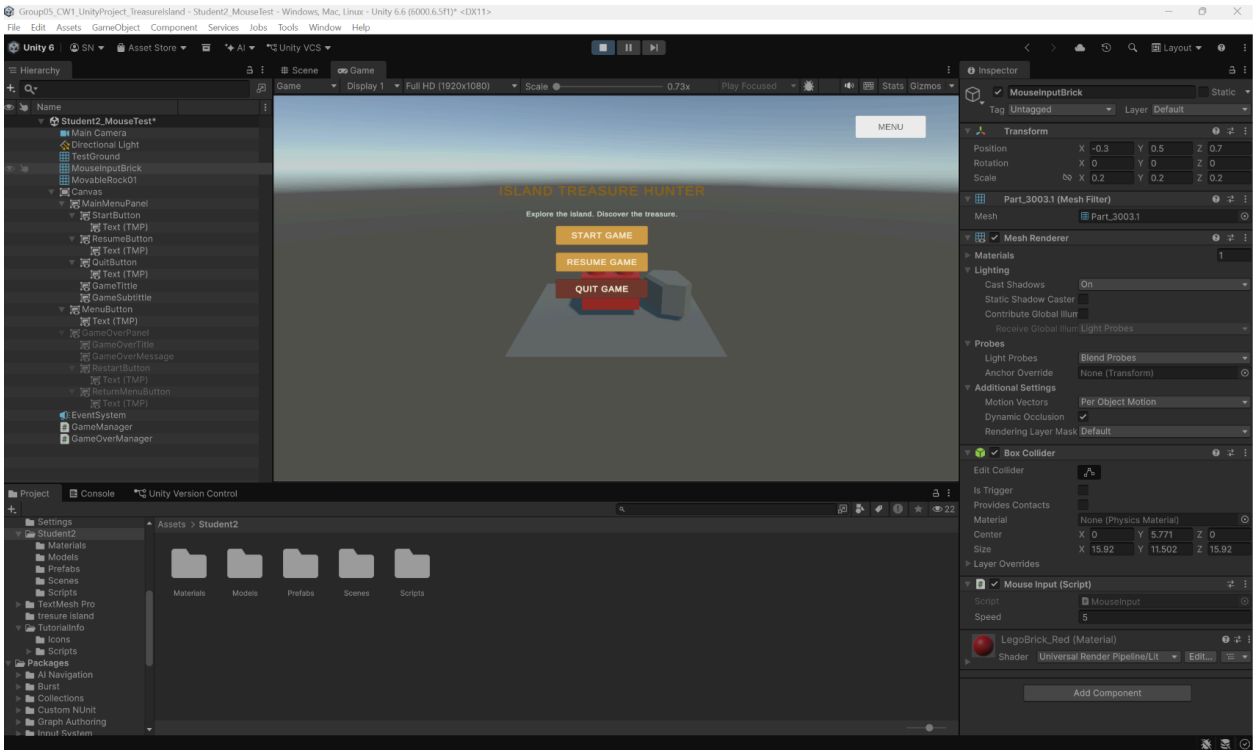
3.2 Direct Mouse Interaction

The project implements direct mouse interaction to allow players to select and manipulate LEGO-style objects within the 3D environment. During CW1, the MouseInput.cs script was developed and tested using a movable LEGO brick and rock. Unity's OnMouseDrag() method detects dragging actions, while Input.GetAxis("Mouse X") and Input.GetAxis("Mouse Y") control object movement. Colliders allow objects to receive mouse events through Unity's camera-based picking system, which uses ray-casting internally. This represents a desktop-based 3D selection and manipulation technique. However, selecting small or distant objects can be difficult, and cursor locking during third-person gameplay requires additional interaction handling.

3.3 2D HUD

The project uses Unity's Canvas system to develop a 2D Heads-Up Display (HUD) that allows players to access essential gameplay controls. During CW1, a menu interface was created with Start, Resume and Quit buttons, along with a separate Game Over panel. These UI elements were designed using a Screen Space Canvas, allowing them to remain visible independently of the camera's position in the 3D environment. The GameMenuController.cs script manages menu visibility and pause functionality using Time.timeScale. For CW2, the group plans to improve the HUD by adding treasure collection indicators, gameplay instructions and additional feedback to enhance the overall player experience.


GAME OVER!
Oops, You fell into the river!
RESTART THE GAME.
MENU


ISLAND TREASURE HUNTER
Explore the island. Discover the treasure.
START GAME
RESUME GAME
QUIT GAME
MENU

3.4 Keyboard State Changes

Keyboard interaction is designed to support character navigation and changes between different gameplay states in Island Treasure Hunter. During CW1, the ThirdPersonPlayerController.cs script was prepared to support character movement using the W, A, S and D keys, while the ThirdPersonMouseCamera.cs script was developed for camera control. The menu system was also designed to allow players to pause and resume gameplay using keyboard input and UI controls. When the pause menu opens, the cursor should become visible and gameplay should stop. During CW2, these controls will be integrated with the final LEGO minifigure and tested within the complete island environment.

Key	Action	State change / result
Input	Open/close pause menu and release/lock cursor	Code prepared; test in Play mode
Esc	Camera-relative LEGO character movement	Controller prepared; final scene pending
WASD	Orbit third-person camera while cursor locked	Camera script prepared; final scene pending
Mouse movement	Open/close pause menu and release/lock cursor	Code prepared; test in Play mode

3.5 Triggers and Collisions

Triggers and collisions are used to detect player interactions with environmental objects and activate gameplay events. During CW1, a cave entrance trigger was developed using a GameObject with a Box Collider configured with the Is Trigger option. The CaveTrigger.cs script detects when the player enters the trigger area by checking the Player tag. A breakable stone wall was also prepared using individual LEGO-style stone objects with colliders and Rigidbody components. The WallExplosion script was intended to activate the wall-breaking effect. Although the cave trigger detection was tested, the complete wall explosion requires further verification and integration during CW2.

3.6 Interaction Summary

Interaction	Input	Feedback	Status
Move LEGO brick / rock	Mouse drag	Object changes position	CW1 tested
Start / Resume / Quit	UI click	Menu visibility / time scale	Created; verify
Third-person movement	WASD + mouse	Movement / camera orbit	CW1 tested
Cave entrance	Player enters trigger	Console detection	Member 4 reports test
Breakable wall	Trigger / physics	Pieces scatter	Not confirmed
Treasure discovery	Planned interaction	Victory feedback	CW2 planned

3.7 Planned Extensions for CW2

During CW2, the group plans to further develop Island Treasure Hunter by integrating the individual components created during CW1 into a complete interactive gameplay experience. The existing object manipulation system will be improved to support puzzle-solving activities, while the cave trigger and breakable stone wall will be integrated into the treasure-hunting storyline. Additional features, including treasure collection, victory animations, improved UI feedback and sound effects, are planned to make the gameplay more engaging. Further testing will focus on collision detection, character movement, camera control and smooth interaction between different gameplay components.

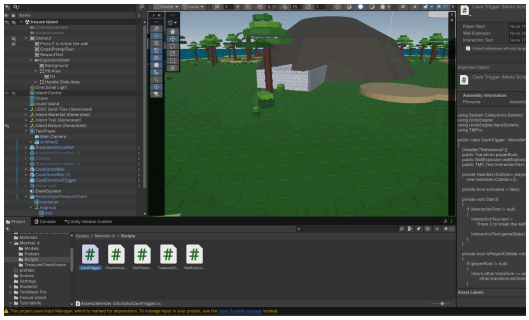
4. Animation, Movement, Narrative

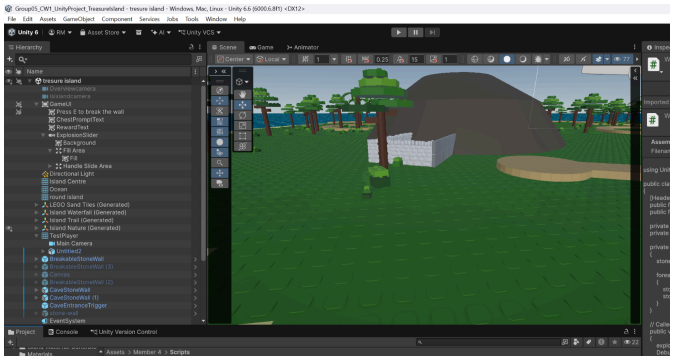
4.a. Figure and Non-Figure Animation, Triggered and Narrative

CW1 – Existing work	CW2 – Planned improvements
Imported a LEGO character and worked on walking and standing animations.	Create original LEGO character models and animations using Blender or 3ds Max, as required by CW2.
Created a cave entrance trigger that responds when the player presses E.	Improve the cave puzzle with additional animation and visual effects.
Created a breakable stone wall using Rigidbody physics to scatter the stones.	Improve the wall-breaking animation and add sound effects.
Set up a treasure chest with an opening animation (Take 001).	Refine the chest-opening animation and reveal gold coins inside the chest.
Added an on-screen congratulations message for finding the treasure.	Develop a more complete treasure story with challenges and rewards.

4.(b) Transforms and Scripted Movement or Interaction

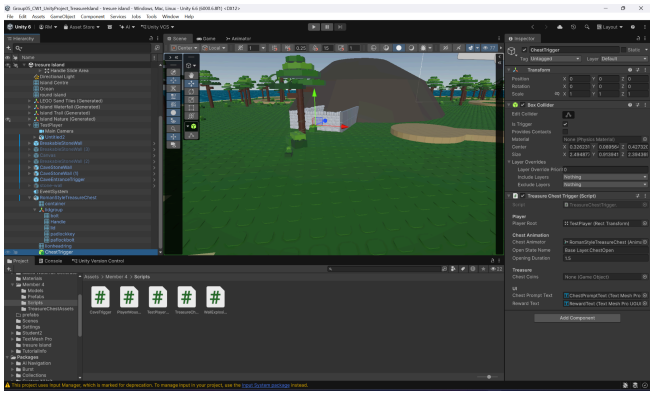
CW1 – Current implementation	CW2 – Planned improvements
Implemented WASD character movement and Spacebar jumping.	Improve player movement and jumping reliability.
Used camera-relative movement to control the character while looking around.	Refine third-person camera controls.
Used **CaveTrigger.cs** to detect the player and the E-key interaction.	Introduce more interactive puzzles and obstacles.
Used **WallExplosion.cs** to scatter individual wall stones through physics.	Improve the explosion effect and potentially allow adjustable explosion force.
Created **TreasureChestTrigger.cs** to control the chest interaction and animation.	Add more treasure discovery effects and rewards.

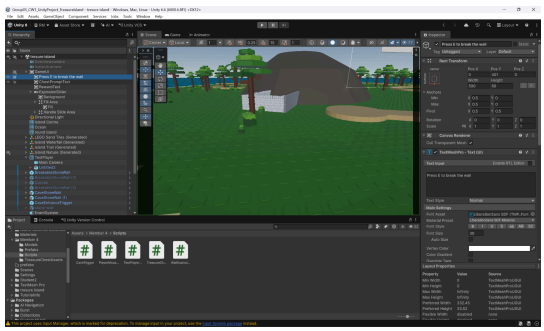




4(c) Investigate Unity Options

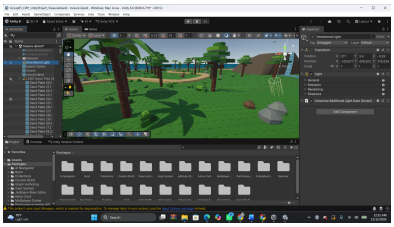
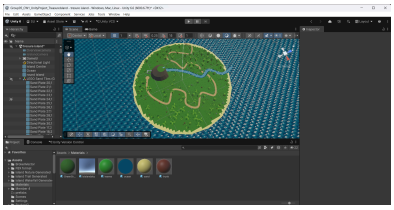
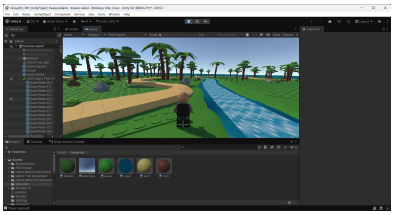
Unity feature	How it relates to our work
Animator and Animation Clips	LEGO walking animation and treasure chest opening
Rigidbody Physics	Exploding brick array
Box Collider and Trigger Events	Cave entrance and treasure chest interaction
Input System	WASD, Spacebar and E-key controls
Unity UI and TextMeshPro	“Press E to break the wall”, “Press E to open chest” and congratulations messages
UI Slider	Planned explosion-force control, if assigned to your group
Prefabs	Reusing and integrating Member 4 objects





5. Lighting, Shaders, and Texturing

Island Treasure Hunter uses Unity's Universal Render Pipeline (URP). The current scene uses plain-colour materials to give the island a LEGO-style appearance. For CW2, we plan to upgrade the environment and build a cave to house the treasure chest.

#	Provisional approach	Evidence to capture
Lighting	The scene currently uses a Directional Light, and the placed objects cast shadows. For CW2, we propose testing the lighting at the cave entrance and inside the cave so the player can see the route and identify the treasure chest.	
Shaders and materials	The beach sand, grass, rocks, trees and bushes use plain colours and LEGO-style shapes. For the environment upgrade, we propose testing **URP Lit** materials with different smoothness settings to distinguish shiny LEGO bricks from matte sand and rock.	
Water	The sea and river currently share one material with visible waves. For CW2, we propose testing whether the waves and water colour keep the riverbank and crossing easy to identify from the third-person camera.	

Unity Boat Attack demonstrates animated waves in Unity's Universal Render Pipeline. It is a reference for improving the waves around the island in CW2. https://github.com/Unity-Technologies/BoatAttack.

6. Project Management Methodology

Selected methodology: Agile (iterative development)

Project Management Methodology – Agile

Our group followed an Agile-inspired methodology to develop the Treasure Island Unity project. Agile is a flexible project management approach that focuses on teamwork, continuous improvement, and developing a project through small stages. We chose this approach because it allowed us to develop different parts of the project separately, test them, and make changes when necessary instead of completing the entire project at once.

At the beginning, we divided the coursework requirements among our four group members and assigned individual responsibilities. As the group leader, I created the GitHub repository, invited the team members as collaborators, and managed our development workflow using GitHub Desktop. We used separate branches so each member could work on their assigned tasks without directly affecting the others' work.

Throughout development, we regularly combined our individual contributions into the shared Unity project and tested their functionality. When we encountered problems, such as character movement issues, collision errors, and trigger interactions, we investigated and improved them. This iterative process helped us identify problems early and improve the project step by step.

We also used peer feedback to identify areas for improvement. For Coursework 2, we plan to continue following this Agile approach by developing additional gameplay features, testing them regularly, and making improvements based on feedback.

Overall, this methodology helped our group manage tasks, collaborate effectively, and adapt to challenges during development.

7. Commercial Analysis

Current and Future Requirements

- Current requirements (CW1)

Our group has developed an interactive Unity prototype containing a LEGO style island environment, character movement, a breakable stone wall, keyboard triggered interactions and a treasure chest. These features establish the basic treasure-hunting concept.

- Future requirements (CW2)

Our group plans to develop a more complete interactive experience. Proposed additions include river-crossing challenges, obstacles, player respawning, improved movement and camera controls, sound effects, lighting and more polished animations. CW2 also requires original animated LEGO minifigure work created using Blender or 3ds Max.

Theory

Relevant design theories and principles to discuss:

- User centred design: The game should be understandable and enjoyable for its intended audience.
- Usability and feedback: Players should receive clear information about available actions and whether those actions succeeded.
- Iterative development: The group can improve gameplay through repeated testing and feedback.
- Interaction design: Keyboard controls, mouse movement and on screen instructions should work together logically.

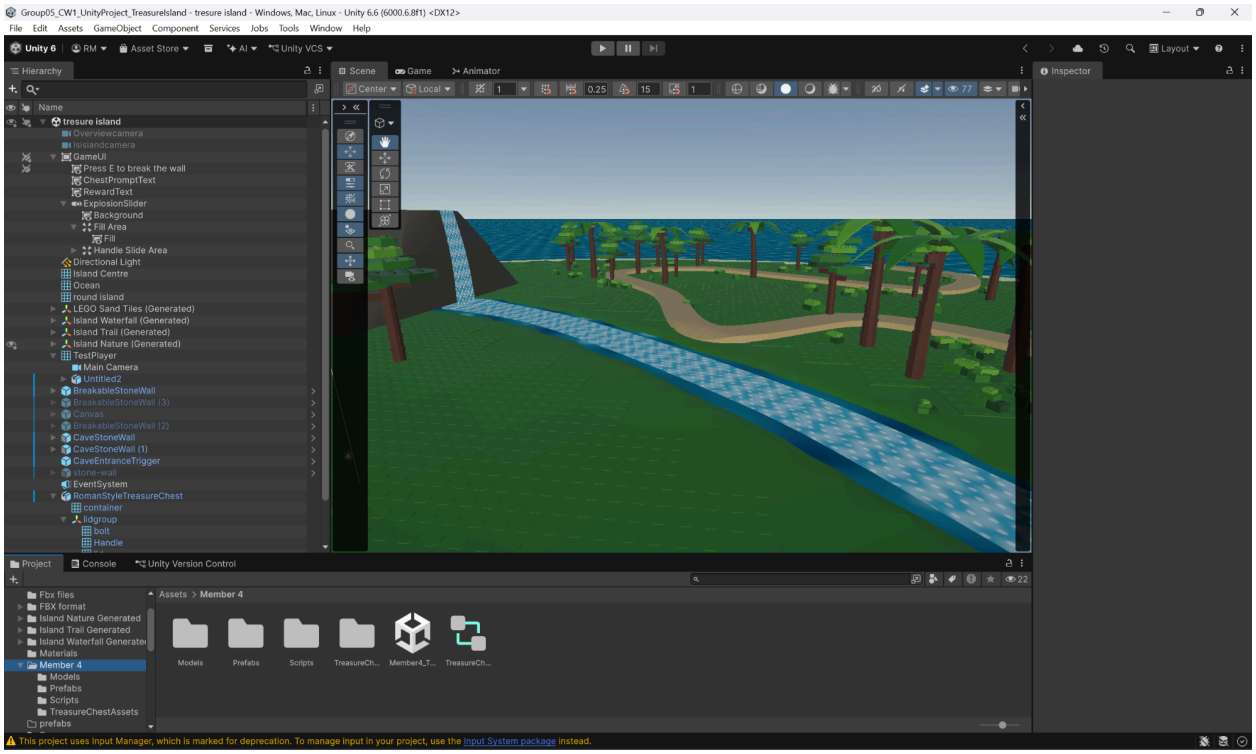
For example, the message “Press E to break the wall” communicates the available action before the player interacts with the cave.

Commercial Issues

Consider the practical issues that would affect the project if it were developed beyond coursework.

Commercial issue	Relevance to Treasure Island
Development cost	Unity and reusable assets help reduce development time, but creating original models and animations requires additional effort
Asset licensing	Downloaded models, textures and animations must have licences permitting their intended use
Intellectual property	LEGO branding and third-party models need appropriate rights if the project is publicly distributed
Performance	Complex models, Rigidbody physics and effects may affect gameplay performance
Maintenance	The group must manage bugs, updates, compatibility and asset dependencies
Potential distribution	Platforms such as itch.io could be investigated for a future demo, subject to licensing and further development

Commercial conclusion to develop: Island Treasure Hunter is currently an educational prototype, not a commercially released game. Before any public release, the group would need to review asset rights, improve reliability and performance, and test whether players find the experience enjoyable.



8. AI Code Assist

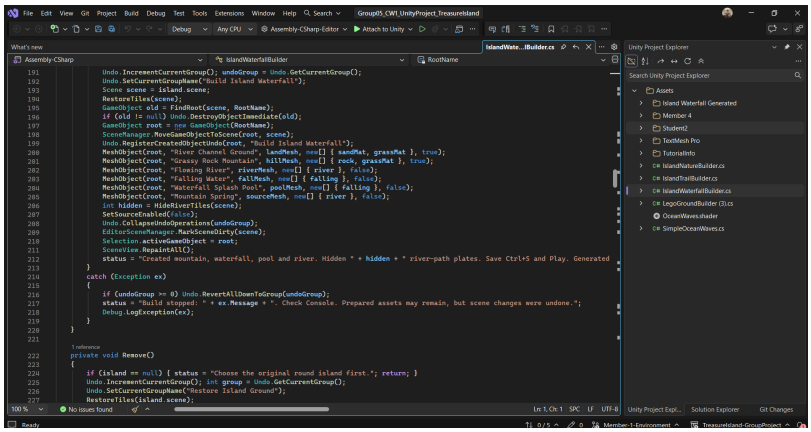
8.1 Responsible use of AI to enhance learning

AI coding assistants can help us to understand programming concepts, identify errors and explore possible solutions. During CW1, ChatGPT was used to explain Unity C# methods such as OnMouseDrag() and Input.GetAxis(), supporting the development of the LEGO brick mouse interaction system. AI guidance also helped investigate an Input System compatibility error. The suggested changes were checked independently through Unity Play mode, where mouse dragging was tested using a LEGO brick and a rock. This experience demonstrated the importance of understanding code rather than simply copying suggestions.

Evidence from Student 1's CW1 work

My responsibility in Island Treasure Hunter is to develop the LEGO-style island environment and implement the LEGO Brick Rigidbody Physics Controller. So far, I have shaped the island and coastline, built the LEGO-style beach, ocean and grass areas, and added the ocean, mountain, waterfall, river, rocks, trees and bushes.
I used plain colour materials to give the environment a consistent LEGO look, added a Directional Light to cast shadows, and used the same animated URP water material for the sea and river. I also laid out the

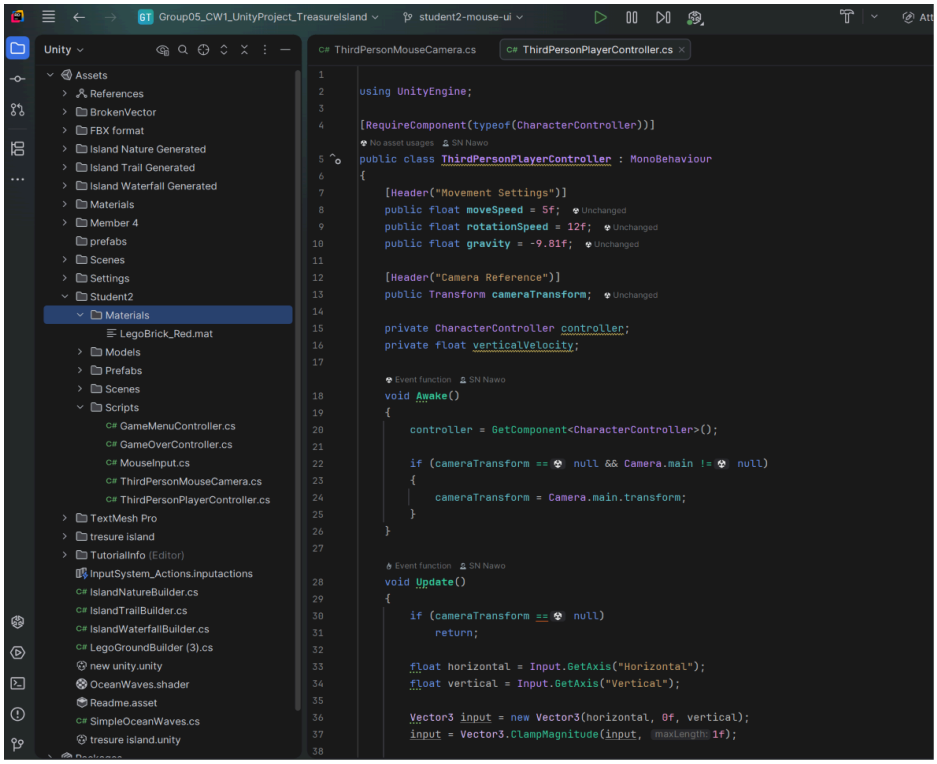
exploration route from the beach through the forest towards the river and mountain, leaving enough space for the player to move and jump. My next task is to build LEGO stepping stones and add obstacles across the river. For the Rigidbody controller, I plan to add a Rigidbody and a Collider to one LEGO brick and write a script that applies a force when the player lands on it. I will test how the brick moves, how it collides with the player, and whether the mission fails as expected if the player falls into the river
I set up a playable LEGO minifigure in Unity. The player uses the W, A, S and D keys to move around the scene and the space bar to jump. The player character currently appears as a capsule, and its movement and jump controls are functional.



Evidence from Student 2's CW1 work

- Unity mouse interaction: investigated OnMouseDrag(), colliders and mouse input axes in MouseInput.cs.
- Debugging: an Input System exception was addressed by changing Active Input Handling to Both.
- Verification: tested dragging a LEGO brick and movable rock in the Student2_MouseTest scene during Play mode.
- Third-person controller and camera scripts were prepared; do not claim successful final-scene integration without testing.

During CW1, I used ChatGPT to improve my understanding of Unity C# scripting and troubleshooting problems related to mouse interaction. I explored how OnMouseDrag(), colliders and mouse input axes work together in the MouseIrick and rock dragging in the Student2_MouseTest scene during Play mode. Additionally, I prepared third-person controller and camera scripts, although their final integration still requires testing. This experience helped me understand Unity input handling and the importance of evaluating AI suggestions through practical testing.



Evidence from Student 3's CW1 work

I contributed to the development of the playable LEGO character in our Island Treasure Hunter project. His work included creating the character in Mecabricks, importing FBX animation files into Unity, and developing the TestPlayerMovement.cs script for character movement and jumping. I also configured Rigidbody physics, CapsuleCollider components and the Animator Controller for standing, walking and sleeping animations.

These tasks involved technical challenges related to character movement, ground detection and animation transitions. AI assistance could be useful for understanding Unity C# concepts and troubleshooting these issues, but any suggested solutions would need to be reviewed and tested in Unity.

Evidence from Student 4's CW1 work

Student 4 contributed to the development of the interactive cave puzzle in the *Island Treasure Hunter* Unity project. The work included creating the cave entrance trigger using CaveTrigger.cs, implementing

the **“Press E to break the wall”** interaction, and developing a breakable stone wall using Rigidbody physics and WallExplosion.cs.

Additional contributions included a treasure chest interaction using TreasureChestTrigger.cs, TextMeshPro instructions, a chest-opening animation setup, and a congratulations message. Student 4 also supported LEGO character movement and collision testing, organised reusable prefabs, and managed the group's GitHub repository, including collaborators, branches and integration.

8.2 AI support for stretch targets beyond tutorial code

For CW2, our group plans to use AI assistance to explore advanced features beyond the basic Unity tutorial requirements. The main stretch goals include improving third-person camera controls, implementing interactive treasure collection, and developing a cave puzzle with breakable stone walls. AI tools such as ChatGPT may help us understand how to switch between camera movement and mouse-based object manipulation, detect treasure collection events, and connect trigger events with animations and physics. Each suggested solution will be reviewed and tested independently in Unity before being integrated into the main project. These improvements are planned for CW2 and will require further development, debugging, and testing to ensure smooth gameplay.

CW2 extension	Appropriate AI support	Current Status
Camera and cursor interaction	Help compare approaches for camera lock vs free cursor for dragging	Integration pending
Cave puzzle and breakable wall	Explain trigger debugging and safe test cases	Trigger tested: wall effect verified
Treasure collection and victory feedback	Help plan component responsibilities and tests	Planned
HUD and UI improvements	Explain Canvas options and UI troubleshooting	Basic menu created; further work planned
Character movement and animation	Explain Unity Animator and movement integration issues	Animation tested

8.3 Critique of AI generated Solutions

AI assistance was useful for understanding Unity C# scripts and identifying possible solutions to development problems. However, the suggested solutions were not always directly suitable for our project. Some required changes to match our GameObjects, colliders, Input System settings and existing scripts.

For example, the mouse interaction code required adjustments to work with Unity's Input System configuration. AI suggestions also needed to be tested carefully because a script that works independently may not function correctly when integrated with other gameplay components. Therefore, we used AI-generated suggestions as learning and debugging references rather than accepting them without checking. We reviewed the logic, made necessary modifications and tested the results in Unity. This helped us understand the limitations of AI assistance and the importance of practical testing, teamwork and independent problem-solving

8.4 Sustainability, Trade-offs and Maintainability Review

During CW1, our group considered code efficiency, maintainability and long-term sustainability when evaluating AI-assisted solutions. Using separate C# scripts for mouse interaction, character movement, camera control and UI management helps keep the project organised and makes future modifications easier. The OnMouseDrag() method provides a simple approach to object manipulation, although a raycasting-based system could offer greater flexibility for advanced interactions in CW2. However, introducing more complex systems may increase development time and debugging requirements. To maintain performance, unnecessary calculations and excessive physics operations should be avoided. AI-generated suggestions must also be reviewed for readability, compatibility and efficiency. These considerations will help our group develop a more reliable, maintainable and expandable Unity project during CW2.

Area	What to discuss
Sustainability	Avoid unnecessary scripts, repeated calculations and excessive physics processing
Trade-offs	OnMouseDrag() is simple to implement but offers less flexibility than a custom ray-casting interaction system
Maintainability	Keep mouse input, camera, movement and menu functionality in separate scripts
Code quality	Use meaningful variable names, comments and consistent coding conventions
Future improvements	Test scripts together and reduce conflicts between camera and object interactions

9. Discuss UI Efficacy

UI efficacy means how effectively the game's controls and interface help the player complete tasks. The following points match the four subheadings in your CW1 brief.

Conceptual Control to Task Mapping

This explains whether the controls match the actions players expect.

Control	Task	Why it is suitable
W, A, S, D	Move the LEGO character	Familiar controls for PC games
Mouse	Look around the island	Helps players explore the 3D environment
Spacebar	Jump	Commonly recognised jumping control
E	Break the wall or open the chest	One consistent interaction key for nearby objects
UI Slider	Adjust explosion force, if implemented	Provides direct control over a numerical value

Using the same E key for both the cave wall and treasure chest creates consistency. The player only needs to learn one interaction key.

Perceptual Structure, Cognitive Load and Disconnect

This section concerns how easily players can see, understand and respond to the interface.

Points to discuss:

- TextMeshPro messages display the available interaction directly in the Game view.
- “Press E to break the wall” and “Press E to open chest” provide instructions at the relevant locations.
- The congratulations message gives feedback after discovering the treasure.
- Simple instructions reduce the number of controls players must remember.
- A possible disconnect occurs when an object appears interactive but no message is displayed, or when pressing E produces no visible response.

CW2 improvement: Use clearer prompt positioning, consistent fonts and colours, and better visual and sound feedback.

Motor Coordination and Any Tiring or Clash

This examines how physically comfortable the controls are and whether they interfere with each other.

Points to discuss:

- WASD and mouse movement allow the player to navigate using both hands.
- Spacebar is easy to access while using WASD.
- The E key is close to WASD, making interactions convenient.
- Repeated camera movement or difficult jumping sections could become tiring.
- Camera-relative movement problems may cause confusion when the player changes viewing direction.
- Pressing E near overlapping triggers could cause conflicts if multiple actions are available simultaneously.

CW2 improvement: Improve camera movement, prevent conflicting interactions and adjust obstacle difficulty through testing.

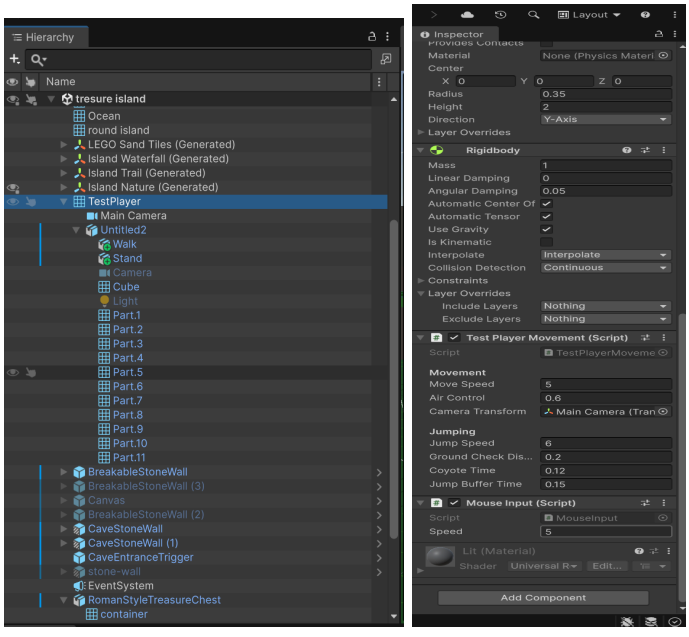
UI Sentiment

UI sentiment concerns how players feel about the interface, including its clarity, comfort, responsiveness and enjoyment.

Points to discuss:

- Simple controls are intended to make the game welcoming to new players.
- Clear E-key messages reduce uncertainty about what to do next.
- The wall-scattering effect provides satisfying visual feedback.
- The treasure chest and congratulations message create a sense of achievement.
- Bugs involving camera movement, jumping or trigger detection could cause frustration.

CW2 improvement: Collect player feedback through testing and use it to improve control responsiveness, message clarity and overall satisfaction. Do not claim positive user-testing results unless your group has actually gathered them



10. Individual Roles

10.1 Suggested Roles

a. Assets: Asset sourcing and conversion

b. Assets: Asset creations

c. Management: Versioning and file management

d. Management: Presentations and milestone updates

e. Management: Meetings, Admin, Schedule

f. Design: Aesthetic design and design bible

g. Design: Interaction design and diagrams

h. Design: Concept and requirements

i. Coding: Interaction

j. Coding: Movement and camera

k. Coding: Animation and trigger

l. Animation: Animation and Implementation

m. Animation Coding: Scripted movement

Student	Selected Roles
Student 1	f. Design: Aesthetic design and design bible i. Coding: Interaction
Student 2	g.Design: Interaction design and diagrams. j. Coding: Movement and camera
Student 3	b. Assets: Asset creation g. Design: Interaction design and diagrams
Student 4	e. Management: Meetings, Admin, Schedule k. Coding: Animation and trigger